

Program 4

Title: Implementing Composite Design Pattern in Java

Problem Description: The above pattern can be implemented to represent the computer filesystem which is composition of files and directories.

Method :

1. Component (FileSystemComponent): Defines a common interface for both files and directories, including operations like display() and getSize().
2. Leaf (File): Represents individual files, implementing FileSystemComponent and providing actual file behavior.
3. Composite (Directory): Represents a directory that can contain multiple FileSystemComponent objects (both File and Directory). It implements methods to add/remove components and delegates operations to its children.
4. Client (Main class): Uses the composite structure to treat files and directories uniformly

```
import java.util.*;

// Component
interface FileSystemComponent {
    void display();
    int getSize();
}

// Leaf
class File implements FileSystemComponent {
    String name;
    int size;

    File(String name, int size) {
        this.name = name;
        this.size = size;
    }

    public void display() {
        System.out.println("File: " + name + " (" + size + " KB)");
    }
}
```

```

    }

    public int getSize() {
        return size;
    }
}

// Composite
class Directory implements FileSystemComponent {
    String name;
    List<FileSystemComponent> list = new ArrayList<>();

    Directory(String name) {
        this.name = name;
    }

    void add(FileSystemComponent obj) {
        list.add(obj);
    }

    public void display() {
        System.out.println("Directory: " + name);
        for (FileSystemComponent f : list) {
            f.display();
        }
    }

    public int getSize() {
        int total = 0;
        for (FileSystemComponent f : list) {
            total += f.getSize();
        }
    }
}

```

```
    }  
    return total;  
}  
}  
  
// Main class  
public class Main {  
    public static void main(String[] args) {  
  
        File f1 = new File("a.txt", 10);  
        File f2 = new File("b.txt", 20);  
  
        Directory dir = new Directory("My Folder");  
        dir.add(f1);  
        dir.add(f2);  
  
        dir.display();  
  
        System.out.println("Total Size: " + dir.getSize() + " KB");  
    }  
}
```

OUTPUT

Directory: My Folder

File: a.txt (10 KB)

File: b.txt (20 KB)

Total Size: 30 KB

The **Composite Pattern** is used when you want to treat **individual objects and groups of objects in the same way**.

👉 In short:

“Treat single objects and collections of objects uniformly.”

Example

1. Think of a **file system**:

- A **File** is a single object
- A **Folder (Directory)** can contain files *and* other folders

But both can be treated the same way (e.g., open, delete, display).

2. Component → Shape

Leaf → Circle, Square

Composite → Drawing (contains shapes)

Calling draw() on:

- Circle → draws circle
- Drawing → calls draw() on all shapes inside it

Key Idea

Create a **common interface** for:

- **Leaf objects** (single elements)
- **Composite objects** (groups of elements)

So, the client **doesn't need to care** whether it's dealing with one object or many.

Structure

1. **Component (Interface / Abstract class)**

- Defines common operations (like display())

2. Leaf (Single object)

- Represents individual elements
- Example: File

3. Composite (Group object)

- Contains multiple components (leaf or composite)
- Example: Folder

4. Client

- Uses the component interface without worrying about type

Working

- ✓ Composite stores child objects
- ✓ When an operation is called:
 - Leaf → performs action directly
 - Composite → forwards action to its children

When to Use

- When objects form a **tree structure**
- When you want to treat **part and whole the same way**
- Example: UI components, file systems, organization hierarchy

Advantages

- Simplifies client code (no type checking)
- Easy to add new components
- Represents hierarchical structures clearly

Disadvantages

- Can make design too general
- Hard to restrict certain components